

# ROS2 basics

## 1 ROS2 project structure

### 1.1 colcon workspace

The colcon workspace is the directory where software packages are created, modified, and compiled. Colcon's workspace can be intuitively described as a warehouse, which contains various ROS project projects to facilitate system organization, management and calling.

- **Create workspace:**

```
mkdir -p ~/colcon_ws/src # Create folder
cd ~/colcon_ws/ # Enter the folder
colcon build # Build the code in the workspace.
```

**Note:** colcon supports option `--symlink-install`. This allows for faster iteration by changing installed files by changing files in the source space (such as Python files or other uncompiled resources). Avoid the need to recompile every time you modify your python script.

```
colcon build --symlink-install
```

A ROS workspace is a directory with a particular structure. Commonly there is a `src` subdirectory. Inside that subdirectory is where the source code of ROS packages will be located. Typically the directory starts otherwise empty.

colcon does out of source builds. By default it will create the following directories as peers of the `src` directory:

```
src/: colcon package for ROS2 (source code package)

build/: The location where intermediate files are stored. For each package, a
subfolder is created in which CMake is called, for example.

install/: The installation location of each package. By default, each package
will be installed into a separate subdirectory.

log/: Contains various logging information about each colcon call.
```

The directory structure of a ROS2 workspace is as follows:

```
workspace --- Customized workspace.
  |-- build: The directory where intermediate files are stored. A separate
  subdirectory will be created for each function package in this directory.
  |-- install: Installation directory, a separate subdirectory will be
  created for each function package in this directory.
  |-- log: Log directory, used to store log files.
  |-- src: Directory used to store function package source code.
  |-- C++ function package
  |-- package.xml: package information, such as: package name,
  version, author, dependencies.
```

```
|-- CMakeLists.txt: Configure compilation rules, such as source
files, dependencies, and target files.
|-- src: C++ source file directory.
|-- include: header file directory.
|-- msg: message interface file directory.
|-- srv: Service interface file directory.
|-- action: action interface file directory.
|-- Python function package
|-- package.xml: package information, such as: package name,
version, author, dependencies.
|-- setup.py: similar to CMakeLists.txt of C++ function package.
|-- setup.cfg: Function package basic configuration file.
|-- resource: resource directory.
|-- test: stores test-related files.
|-- Directory with the same name of the function package: Python
source file directory.
```

## 1.2 ROS2 package

Package is not only a software package on Linux, but also the basic unit of colcon compilation. The object we use `colcon build` to compile is each ROS2 package.

### Create your own package:

- The command syntax for creating a software package using Python is:

```
ros2 pkg create --build-type ament_python <package_name>
```

- For example:

```
ros2 pkg create --build-type ament_python --node-name my_node my_package
```

## 2 Basic tool commands

In this chapter, you will learn about the common command tools of ROS2.

### 2.1 Topics

ROS 2 breaks complex systems down into many modular nodes. Topics are a vital element of the ROS graph that act as a bus for nodes to exchange messages. Topics are one of the main ways in which data is moved between nodes and therefore between different parts of the system.

Specific reference: [Official Tutorials](#)

- **topics help**

```
ros2 topics -h
```

- **Start turtlesim and keyboard control**

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

- **Node Relationship Diagram**

```
rqt_graph
```

- **Learn about topic-related commands**

```
ros2 topics -h
```

- **topics list**

```
ros2 topic list
ros2 topic list -t # Display the corresponding message type
```

- **View topic content**

```
ros2 topic echo <topic_name>
ros2 topic echo /turtle1/cmd_vel
```

- **Display topic-related information, type**

```
ros2 topic info <topic_name>
# Output /turtle1/cmd_vel topic related information
ros2 topic info /turtle1/cmd_vel
```

- **Display interface related information**

```
ros2 interface show <msg_type>
# Output geometry_msgs/msg/Twist interface related information
ros2 interface show geometry_msgs/msg/Twist
```

- **Issue an order**

```
ros2 topic pub <topic_name> <msg_type> '<args>'
# Issue speed command
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
# Issue speed commands at a certain frequency
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

- **See how often topics are posted**

```
ros2 topic hz <topic_name>
# Output /turtle1/cmd_vel publish frequency
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

## 2.2 Nodes

---

Each node in ROS should be responsible for a single, module purpose (e.g. one node for controlling wheel motors, one node for controlling a laser range-finder, etc). Each node can send and receive data to other nodes via topics, services, actions, or parameters. A full robotic system is comprised of many nodes working in concert. In ROS 2, a single executable (C++ program, Python program, etc.) can contain one or more nodes.

Specific reference: [Official Tutorials](#)

- **nodes help**

```
ros2 nodes -h
```

- **Start turtlesim and keyboard control**

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

- **View the node list**

```
ros2 node list
```

- **View Node Relationship Diagram**

```
rqt_graph
```

- **Remapping**

```
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle  
ros2 node list
```

- **View node information**

```
ros2 node info <node_name>  
ros2 node info /my_turtle
```

## 2.3 Services

---

Services are another method of communication for nodes in the ROS graph. Services are based on a call-and-response model, versus topics' publisher-subscriber model. While topics allow nodes to subscribe to data streams and get continual updates, services only provide data when they are specifically called by a client.

Specific reference: [Official Tutorials](#)

- **services help**

```
ros2 service -h
```

- **Start turtlesim and keyboard control**

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

- **View the service list**

```
ros2 service list
# Display service list and message type
ros2 service list -t
```

- **View the message types received by the service**

```
ros2 service type <service_name>
ros2 service type /clear
```

- **Find services that use a certain message type**

```
ros2 service find <type_name>
ros2 service find std_srvs/srv/Empty
```

- **View Service Message Type Definitions**

```
ros2 interface show <type_name>.srv
ros2 interface show std_srvs/srv/Empty.srv
```

- **Call the service command to clear the walking track**

```
ros2 service call <service_name> <service_type>
ros2 service call /clear std_srvs/srv/Empty
```

- **Spawn a new turtle**

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 2, y: 2, theta: 0.2, name:
'turtle2'}"
```

## 2.4 Parameters

A parameter is a configuration value of a node. You can think of parameters as node settings. A node can store parameters as integers, floats, booleans, strings, and lists. In ROS 2, each node maintains its own parameters. For more background on parameters, please see the concept document.

Specific reference: [Official Tutorials](#)

- **parameters help**

```
ros2 param -h
```

- **Start turtlesim and keyboard control**

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

- **View service list**

```
ros2 param list
```

- **Get the parameter value**

```
ros2 param get <node_name> <parameter_name>
ros2 param get /turtlesim background_g
```

- **Set parameter values**

```
ros2 param set <node_name> <parameter_name> <value>
ros2 param set /turtlesim background_r 150
```

- **Export parameter values**

```
ros2 param dump <node_name>
ros2 param dump /turtlesim
```

- **Import parameters independently**

```
ros2 param load <node_name> <parameter_file>
ros2 param load /turtlesim ./turtlesim.yaml
```

- **Start the node and import parameters at the same time**

```
ros2 run <package_name> <executable_name> --ros-args --params-file <file_name>
ros2 run turtlesim turtlesim_node --ros-args --params-file ./turtlesim.yaml
```

## 2.5 Actions

Actions are one of the communication types in ROS 2 and are intended for long running tasks. They consist of three parts: a goal, feedback, and a result.

Actions are built on topics and services. Their functionality is similar to services, except actions are preemptable (you can cancel them while executing). They also provide steady feedback, as opposed to services which return a single response.

Actions use a client-server model, similar to the publisher-subscriber model (described in the topics tutorial). An “action client” node sends a goal to an “action server” node that acknowledges the goal and returns a stream of feedback and a result.

Specific reference: [Official Tutorials](#)

- **action help**

```
ros2 action -h
```

- **Start turtlesim and keyboard control**

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

Press G|B|V|C|D|E|R|T to achieve rotation, press F to cancel

- **View the server and client of the node action**

```
ros2 node info /turtlesim
```

- **View action list**

```
ros2 action list
ros2 action list -t # show action type
```

- **view action info**

```
ros2 action info <action>
ros2 action info /turtle1/rotate_absolute
```

- **View action message content**

```
ros2 interface show turtlesim/action/RotateAbsolute
```

- **Send action target information**

```
ros2 action send_goal <action_name> <action_type>
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "
{theta: 1.57}"
# with feedback information
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "
{theta: 0}" --feedback
```

## 2.6 RQt

RQt is a graphical user interface framework that implements various tools and interfaces in the form of plugins. One can run all the existing GUI tools as dockable windows within RQt! The tools can still run in a traditional standalone method, but RQt makes it easier to manage all the various windows in a single screen layout.

Specific reference: [Official Tutorials](#)

You can run any RQt tools/plugins easily by:

```
rqt
```

- **rqt help**

```
rqt -h
```

- **Start turtlesim and keyboard control**

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

- Action Type Browser: / Plugins -> Actions -> Action Type Browser
- parameter reconfiguration: / Plugins -> configuration -> Parameter Reconfigure
- Node graph: /Node Graph
- control steering: /Plugins -> Robot Tools -> Robot Steering
- service invocation: /Plugins -> Services -> Service Caller
- Service Type Browser: Plugins -> Services -> Service Type Browser
- message release: Plugins -> Topics -> Message Publisher
- Message Type Browser: Plugins -> Topics -> Message Type Browser
- topic list: Plugins -> Topics -> Topic Monitor
- draw a graph: Plugins -> Visualization -> Plot
- **View logs: rqt\_console**

```
ros2 run rqt_console rqt_console
ros2 run turtlesim turtlesim_node
ros2 topic pub -r 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0,
y: 0.0, z: 0.0}, angular: {x: 0.0,y: 0.0,z: 0.0}}"
```

## 2.7 TF2

tf2 is the transform library, which lets the user keep track of multiple coordinate frames over time. tf2 maintains the relationship between coordinate frames in a tree structure buffered in time and lets the user transform points, vectors, etc. between any two coordinate frames at any desired point in time.

Specific reference: [Official Tutorials](#)

Let's start by installing the demo package and its dependencies.

```
sudo apt-get install ros-foxy-turtle-tf2-py ros-foxy-tf2-tools ros-foxy-tf-
transformations
```

- **follow**
- launch starts 2 little turtles, the first little turtle automatically follows the second one

```
ros2 launch turtle_tf2_py turtle_tf2_demo.launch.py
```

- Control the movement of the first little turtle through the keyboard

```
ros2 run turtlesim turtle_teleop_key
```

- **View TF tree**



```
ros2 run tf2_tools view_frames.py
evince frames.pdf
```

- **View the relationship between two coordinate systems**

```
ros2 run tf2_ros tf2_echo [reference_frame] [target_frame]
ros2 run tf2_ros tf2_echo turtle2 turtle1
```

- **View TF relationships on rviz**

```
ros2 run rviz2 rviz2 -d $(ros2 pkg prefix --share
turtle_tf2_py)/rviz/turtle_rviz.rviz
```

## 2.8 URDF

URDF is the Unified Robot Description Format for specifying robot geometry and organization in ROS.

Specific reference: [Official Tutorials](#)

- **Complete syntax**

```
<robot>
  # describe:
  # Parameters: name=""
  # Child node:
  <link>
    # Description:
    # Parameters: name=""
    # Child node:
    <visual>
      # describe:
      # Parameters:
      # child nodes:
      <geometry>
        # description
        # parameters
        # Child node:
        <cylinder />
          # Description:
          # Parameters:
            # length="0.6"
            # radius="0.2"
        <box />
          # description
          # Parameters:size="0.6 0.1 0.2"
        <mesh />
          # Description
          #Parameters:
          filename="package://urdf_tutorial/meshes/1_finger_tip.dae"
      <collision>
        # Description: collision element, prioritized
        # parameters
        # child node
```

```

        <geometry>
        <inertial>
            # description
            # parameters
            # Child nodes:
            <mass />
                # description: mass
                # Parameters: value=10
            <inertia />
                # Description: Inertia
                # Parameters: i+"Cartesian product of xyz" (9
in total)="0.4"

        <origin />
            # Description:
            # Parameters:
                # rpy="0 1.5 0"
                # xyz="0 0 -0.3"
        <material />
            # Description
            # Parameters: name="blue"
<joint>
    # Description
    # Parameters:
        # name=""
        # type=""
            # fixed
            # prismatic
    # child node
    <parent />
        # Description
        # Parameters: link=""
    <child />
        # Description:
        # Parameters: link=""
    <origin />
        # Description:
        # Parameters: xyz="0 -0.2 0.25"
    <limit />
        # Description
        # Parameters:
            # effort="1000.0"    maximum effort
            # lower="-0.38"     Joint upper limit (radians)
            # upper="0"         Joint lower limit (radians)
            # velocity="0.5"    Maximum velocity
    <axis />
        # Description: Press ? axis rotation
        # Parameters: xyz="0 0 1", along the Z axis
<material>
    # Description:
    # Parameters: name="blue"
    # child node:
    <color />
        # description:
        # Parameters: rgba="0 0 0.8 1"

```

- **Install dependent libraries**

```
sudo apt install ros-foxy-joint-state-publisher-gui ros-foxy-joint-state-publisher
sudo apt install ros-foxy-xacro
```

- **Download the source code**

```
cd ~/dev_ws
git clone -b ros2 https://github.com/ros/urdf_tutorial.git src/urdf_tutorial
```

- **Compiling the source code**

```
colcon build --packages-select urdf_tutorial
```

- **Running the example**

```
ros2 launch urdf_tutorial display.launch.py model:=urdf/01-myfirst.urdf
```

## 2.9 Launch

The launch system in ROS 2 is responsible for helping the user describe the configuration of their system and then execute it as described. The configuration of the system includes what programs to run, where to run them, what arguments to pass them, and ROS-specific conventions which make it easy to reuse components throughout the system by giving them each a different configuration. It is also responsible for monitoring the state of the processes launched, and reporting and/or reacting to changes in the state of those processes.

Launch files written in Python, XML, or YAML can start and stop different nodes as well as trigger and act on various events.

Specific reference: [Official Tutorials](#)

### Setup

Create a new directory to store your launch files:

```
mkdir launch
```

### Writer the launch file

Let's put together a ROS 2 launch file using the turtlesim package and its executables. As mentioned above.

Copy and paste the complete code into the launch/turtlesim\_mimic\_launch.py file:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
```

```

        namespace='turtlesim1',
        executable='turtlesim_node',
        name='sim'
    ),
    Node(
        package='turtlesim',
        namespace='turtlesim2',
        executable='turtlesim_node',
        name='sim'
    ),
    Node(
        package='turtlesim',
        executable='mimic',
        name='mimic',
        remappings=[
            ('/input/pose', '/turtlesim1/turtle1/pose'),
            ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
        ]
    )
])

```

## Run the ros2 launch file

To run the launch file created above, enter into the directory you created earlier and run the following command:

The syntax format is:

```

ros2 launch <package_name> <launch_file_name>
cd launch
ros2 launch turtlesim_mimic_launch.py

```

- **launch help**

```
ros2 launch -h
```

- **running node**

```
ros2 launch turtlesim multisim.launch.py
```

- **Check the parameters of the launch file**

```

ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py -s
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py --show-arguments
ros2 launch turtlebot3_bringup robot.launch.launch.py -s

```

- **Run the launch file with parameters**

```
ros2 launch turtlebot3_bringup robot.launch.launch.py usb_port:=/dev/opencr
```

- **Run the node and debug**

```
ros2 launch turtlesim turtlesim_node.launch.py -d
```

- **Only output node description**

```
ros2 launch turtlesim turtlesim_node.launch.py -p
```

- **running components**

```
ros2 launch composition composition_demo.launch.py
```

## 2.10 Run

run is used to run a single node, component program

- **run help**

```
ros2 run -h
```

- **running node**

```
ros2 run turtlesim turtlesim_node
```

- **Run node with parameters**

```
ros2 run turtlesim turtlesim_node --ros-args -r __node:=turtle2 -r __ns:=/ns2
```

- **Run component container**

```
ros2 run rclcpp_components component_container
```

- **running components**

```
ros2 run composition manual_composition
```

## 2.11 Package

A package can be considered a container for your ROS 2 code. If you want to be able to install your code or share it with others, then you'll need it organized in a package. With packages, you can release your ROS 2 work and allow others to build and use it easily.

Package creation in ROS 2 uses ament as its build system and colcon as its build tool. You can create a package using either CMake or Python, which are officially supported, though other build types do exist.

Specific reference: [Official Tutorials](#)

### Creating a workspace

Create a new directory for every new workspace. The name doesn't matter, but it is helpful to have it indicate the purpose of the workspace. Let's choose the directory name `ros2_ws`, for "development workspace":

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws/src
```

- **pkg help**

```
ros2 pkg -h
```

- **List Feature Packs**

```
ros2 pkg executable turtlesim
```

- **Output a function package executable program**

```
ros2 pkg executable turtlesim
```

- **Create a Python package**

Make sure you are in the src folder before running the package creation command.

```
cd ~/ros2_ws/src
```

The command syntax for creating a new package in ROS 2 is:

```
ros2 pkg create --build-type ament_python <package_name>  
# you will use the optional argument --node-name which creates a simple Hello  
world type executable in the package.  
ros2 pkg create --build-type ament_python --node-name my_node my_package
```

- **Build a package**

Putting packages in a workspace is especially valuable because you can build many packages at once by running colcon build in the workspace root. Otherwise, you would have to build each package individually.

```
# Return to the root of your workspace:  
cd ~/ros2_ws  
# Now you can build your packages:  
colcon build
```

- **Source the setup file**

To use your new package and executable, first open a new terminal and source your main ROS 2 installation.

Then, from inside the ros2\_ws directory, run the following command to source your workspace:

```
source install/setup.bash
```

Now that your workspace has been added to your path, you will be able to use your new package's executables.

- **Use the package**

To run the executable you created using the `--node-name` argument during package creation, enter the command:

```
ros2 run my_package my_node
```